Pemrosesan Data Time Series Besar

Pertemuan 6 — Analisis Big Data

S1 Teknik Informatika / Sistem Informasi | Semester Ganjil 2026

Dosen Pengampu: Yuda Munarko

Agenda Perkuliahan Hari Ini

1. **Karakteristik & Anatomi Deret Waktu (Time Series)**
   - Perbedaan cross-sectional vs time series, dekomposisi 3 komponen (*Trend, Seasonality, Noise*), dan konsep *Stationarity*.
2. **Resampling & Windowing Temporal di Polars**
   - Agregasi interval kaku (`group_by_dynamic`), rata-rata bergerak (`rolling_mean`), dan perbedaan sliding vs expanding window.
3. **Rekayasa Fitur Temporal & Deteksi Celah (Gaps)**
   - Pembentukan fitur *Lag* & *Lead* (`shift`), *Differencing* (`diff`), kalender, dan penanganan interval sensor yang hilang.
4. **Optimasi Skala Besar: Partisi Parquet & SQL Window Functions**
   - *Hive temporal partitioning* untuk *query pruning*, serta kueri *running total* di DuckDB.
5. **Check-in Tugas 1 (Milestone 3) & Pengumuman Tugas 2 (Phase 1)**

> **Sesi Interaktif Wayground**: Kuis dibagi dalam 3 Checkpoint (Menit 25, Menit 55, dan Menit 85).

Mengapa Data Time Series Berbeda?

Pada data tabular biasa (*cross-sectional*), urutan baris sering kali tidak memengaruhi hasil analisis statistik. Namun pada deret waktu, urutan waktu adalah dimensi fundamental.

Definisi: Time Series Data
Urutan observasi numerik yang dicatat secara kronologis pada interval waktu tertentu (tetap atau bervariasi).

Karakteristik Kunci:

- **Ketergantungan Temporal:** Nilai saat ini (Y_t) sangat dipengaruhi oleh nilai masa lalu (Y_{t-1}).
- **Skala Volume Masif:** Sensor IoT, log server, dan transaksi finansial menghasilkan jutaan rekaman per jam.
- **Frekuensi Data:** Detik (IoT), Menit (Finansial), Jam (Cuaca), Hari (Retail).

Tantangan Khusus Big Data:

- **Sensor Mati / Celah Waktu (*Gaps*):** Data tidak selalu datang tepat waktu dan sering ada interval hilang.
- **Biaya I/O Tinggi:** Membaca jutaan baris timestamp mentah tanpa partisi membebani memori dan disk.
- **Kebocoran Data (*Data Leakage*):** Model analitik tidak boleh memakai data masa depan (*lookahead bias*) saat menghitung agregasi historis.

Dekomposisi Deret Waktu: 3 Komponen Fundamental

Observasi deret waktu (Y_t) secara teoritis dibentuk oleh perpaduan tiga komponen:

1. **Tren Jangka Panjang (*Trend* - T_t):**
   - Arah pergerakan umum data (naik, turun, atau stabil) dalam kurun waktu panjang (bulan/tahun).
2. **Pola Musiman (*Seasonality* - S_t):**
   - Pola variasi yang **berulang secara teratur** pada interval waktu yang konsisten (misal: suhu dingin malam hari, lonjakan transaksi jam makan siang, belanja awal bulan).
3. **Residu Acak (*Noise / Irregular* - I_t):**
   - Fluktuasi acak tanpa pola tetap akibat gangguan cuaca ekstrem atau eror pembacaan sensor.

Model Pembentukan Data:

- **Model Aditif:**

$$Y_t = T_t + S_t + I_t$$

  Digunakan jika amplitudo musiman konstan sepanjang waktu.

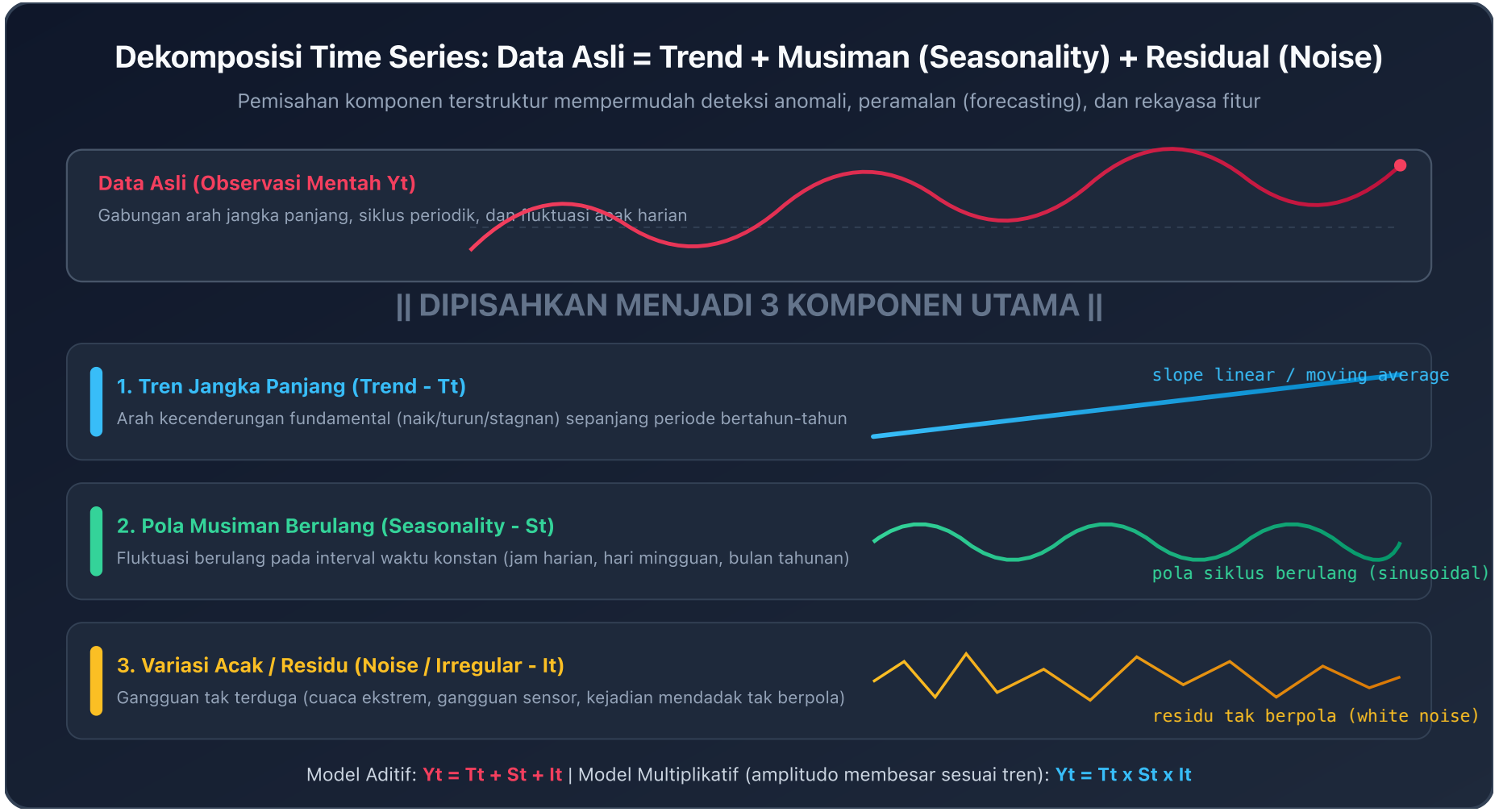
- **Model Multiplikatif:**

$$Y_t = T_t \times S_t \times I_t$$

  Digunakan jika ayunan musiman membesar seiring naiknya tren.

Ilustrasi Visual: Komponen Deret Waktu

Pemisahan data asli menjadi komponen tren, musiman, dan residu mempermudah deteksi anomali.



Istilah Kunci: Resampling & Downsampling

Pada dataset deret waktu skala besar, frekuensi perekaman sering kali terlalu rapat untuk analisis tren jangka panjang (misal: sensor mencatat suhu setiap 1 detik = 86.400 baris/hari).

Metode: Resampling (Downsampling)
Mengubah data berfrekuensi tinggi ke frekuensi yang lebih rendah (misal: dari detik ke jam, atau jam ke harian) menggunakan fungsi agregasi (mean, sum, max, min).

Metode: Resampling (Upsampling)
Mengubah data berfrekuensi rendah ke frekuensi lebih tinggi (misal: data harian ke per jam). Membutuhkan teknik interpolasi untuk mengisi nilai antara.

Tujuan Strategis Downsampling:

1. Mengurangi volume data secara drastis (menghemat 95%+ RAM).
2. Meredam fluktuasi acak (*noise*) sehingga pola tren terlihat jelas.
3. Menyelaraskan frekuensi berbagai sensor yang berbeda interval.

Pertimbangan Praktis:

- Jangan lakukan visualisasi jutaan baris sebelum *downsampling*.
- Selalu pilih fungsi agregasi yang tepat: `SUM` untuk volume transaksi / curah hujan, `AVG` untuk suhu / tekanan.

Istilah Kunci: Stasioneritas (Stationarity)

Dalam analisis time series dan pemodelan prediktif, stasioneritas adalah asumsi paling penting.

Konsep: Stationarity (Stasioner)
Sifat deret waktu di mana parameter statistik dasarnya — terutama **rata-rata (mean)** dan **varians** — bersifat konstan sepanjang waktu, serta tidak memiliki pola musiman (seasonality).

Mengapa Stasioneritas Penting?

- Jika rata-rata terus naik (ada tren), model statistik akan menghasilkan proyeksi yang bias.
- Hubungan antar-variabel hanya valid jika struktur data stabil.

Bagaimana Mengubah Data Non-Stasioner Menjadi Stasioner?

1. **Differencing ($Y_t - Y_{t-1}$):**
   - Menghilangkan tren linier dengan menghitung selisih antar baris (`pl.col("x").diff()`).
2. **Transformasi Logaritma ($\log(Y_t)$):**
   - Menstabilkan varians yang membesar seiring berjalannya waktu.
3. **Seasonal Differencing ($Y_t - Y_{t-s}$):**
   - Menghilangkan efek siklus musiman 24 jam atau 7 hari.

Checkpoint 1: Wayground Interaktif (Menit 25)

Silakan bergabung ke sesi **Wayground** kelas kita untuk Checkpoint 1:

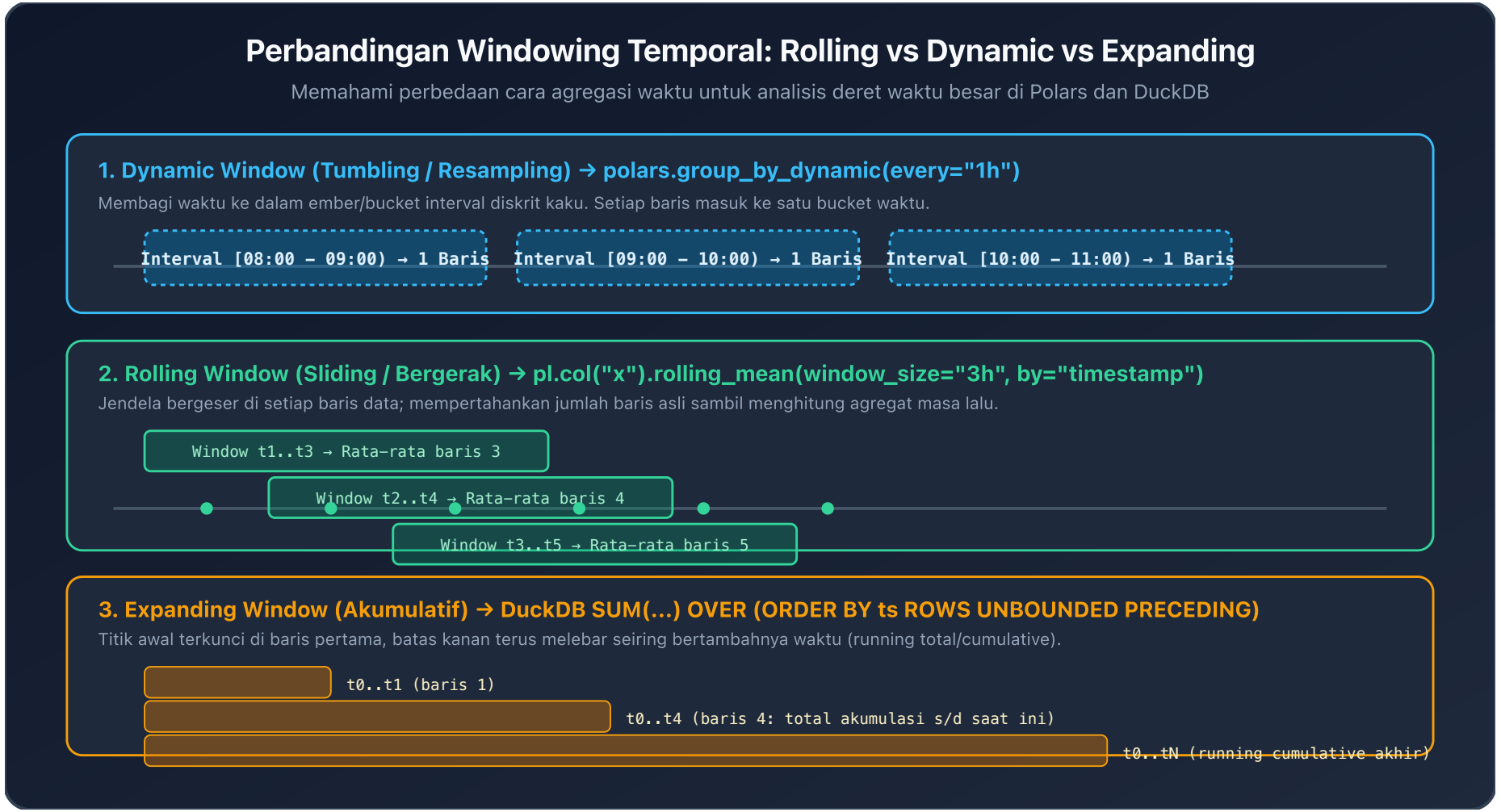
Fokus Evaluasi Checkpoint 1:

- **Soal 1:** Pengenalan komponen deret waktu berulang (*Seasonality* vs *Trend* vs *Noise*).
- **Soal 2:** Tujuan strategis proses *downsampling* pada data berskala besar.
- **Soal 3:** Definisi dan signifikansi matematis dari *Stationarity* pada deret waktu.

> **Aturan Checkpoint:**
>
> - Waktu pengerjaan: 30 detik per soal.
> - Seluruh opsi pilihan memiliki panjang karakter seimbang.
> - Evaluasi hasil akan dibahas langsung bersama dosen sebelum lanjut ke sesi kode.

Operasi Windowing: Rolling vs Dynamic vs Expanding

Dalam pengolahan data temporal, terdapat 3 tipe jendela (*window*) yang sering membingungkan:



Polars: Agregasi Diskrit dengan `group_by_dynamic()`

Untuk melakukan *downsampling* waktu ke dalam ember (*bucket*) interval tetap (misal: per 15 menit atau per 1 jam), Polars menyediakan metode bawaan yang sangat cepat: `group_by_dynamic()`.

```
import polars as pl

# Dataset sensor cuaca BMKG dengan kolom timestamp dan suhu
df_agregat_per_jam = (
    df_sensor
    .sort("timestamp")
    .group_by_dynamic(
        index_column="timestamp",
        every="1h",           # Lebar interval bucket (misal: 1 jam, 15m, 1d)
        period="1h",          # Rentang observasi per window (default sama dengan every)
        closed="left"         # Batas interval: [08:00, 09:00)
    )
    .agg([
        pl.col("suhu").mean().alias("suhu_rata2"),
        pl.col("suhu").max().alias("suhu_maks"),
        pl.col("kelembaban").mean().alias("kelembaban_rata2"),
        pl.len().alias("jumlah_bacaan_sensor")
    ])
)
```

> **Keunggulan Polars:** Algoritma multithreaded di Rust mengeksekusi pengelompokan waktu hingga 50x lebih cepat dibanding `resample()` pada Pandas.

Polars: Rata-Rata Bergerak dengan `rolling()`

Berbeda dengan `group_by_dynamic()` yang memperkecil baris data, `rolling()` mempertahankan seluruh baris data dan menghitung statistik lokal pada jendela bergerak (*sliding window*).

```
# Menghitung moving average 7 hari dan standar deviasi bergerak
df_fitur = df_harian.with_columns([
    # Rata-rata bergerak 7 hari berdasarkan kolom tanggal
    pl.col("harga")
    .rolling_mean(window_size="7d", by="tanggal")
    .alias("harga_ma_7d"),

    # Standar deviasi bergerak 7 hari untuk mengukur volatilitas
    pl.col("harga")
    .rolling_std(window_size="7d", by="tanggal")
    .alias("harga_volatilitas_7d"),

    # Nilai ekstrem maksimum dalam jendela 30 hari ke belakang
    pl.col("harga")
    .rolling_max(window_size="30d", by="tanggal")
    .alias("harga_puncak_30d")
])
```

> **Kapan Digunakan?** Menghaluskan fluktuasi harian (*smoothing*), mendeteksi anomali (*outlier* lokal saat $Y_t > \mu_{7d} + 3\sigma_{7d}$), dan indikator teknikal.

Feature Engineering Temporal: Lag & Lead di Polars

Dalam pemodelan data deret waktu, nilai historis masa lalu adalah prediktor terbaik untuk masa kini.

Metode: Lag Feature (.shift(k))

Menggeser nilai kolom ke masa lalu sebanyak k langkah. Baris saat ini akan berisi nilai dari $t-k$.

```
# Selisih hari ini vs kemarin (Lag 1)
df_lag = df.with_columns([
    # Nilai 1 langkah sebelumnya
    pl.col("curah_hujan")
    .shift(1)
    .alias("hujan_kemarin"),

    # Perubahan nilai (differencing)
    (pl.col("curah_hujan") -
     pl.col("curah_hujan").shift(1))
    .alias("delta_hujan")
])
```

Metode: Kalender & Musiman

Ekstraksi komponen waktu dari timestamp menjadi variabel kategorikal/numerik pendukung.

```
df_kalender = df.with_columns([
    pl.col("ts").dt.hour().alias("jam"),
    pl.col("ts").dt.weekday().alias("hari"),
    pl.col("ts").dt.month().alias("bulan"),
    (pl.col("ts").dt.weekday() >= 6)
    .alias("is_weekend")
])
```

> **Ingat:** Jangan pernah gunakan `shift(-k)` (Lead) sebagai prediktor karena akan menyebabkan kebocoran data masa depan!

Checkpoint 2: Wayground Interaktif (Menit 55)

Waktunya menguji pemahaman sintaks dan operasi temporal Polars:

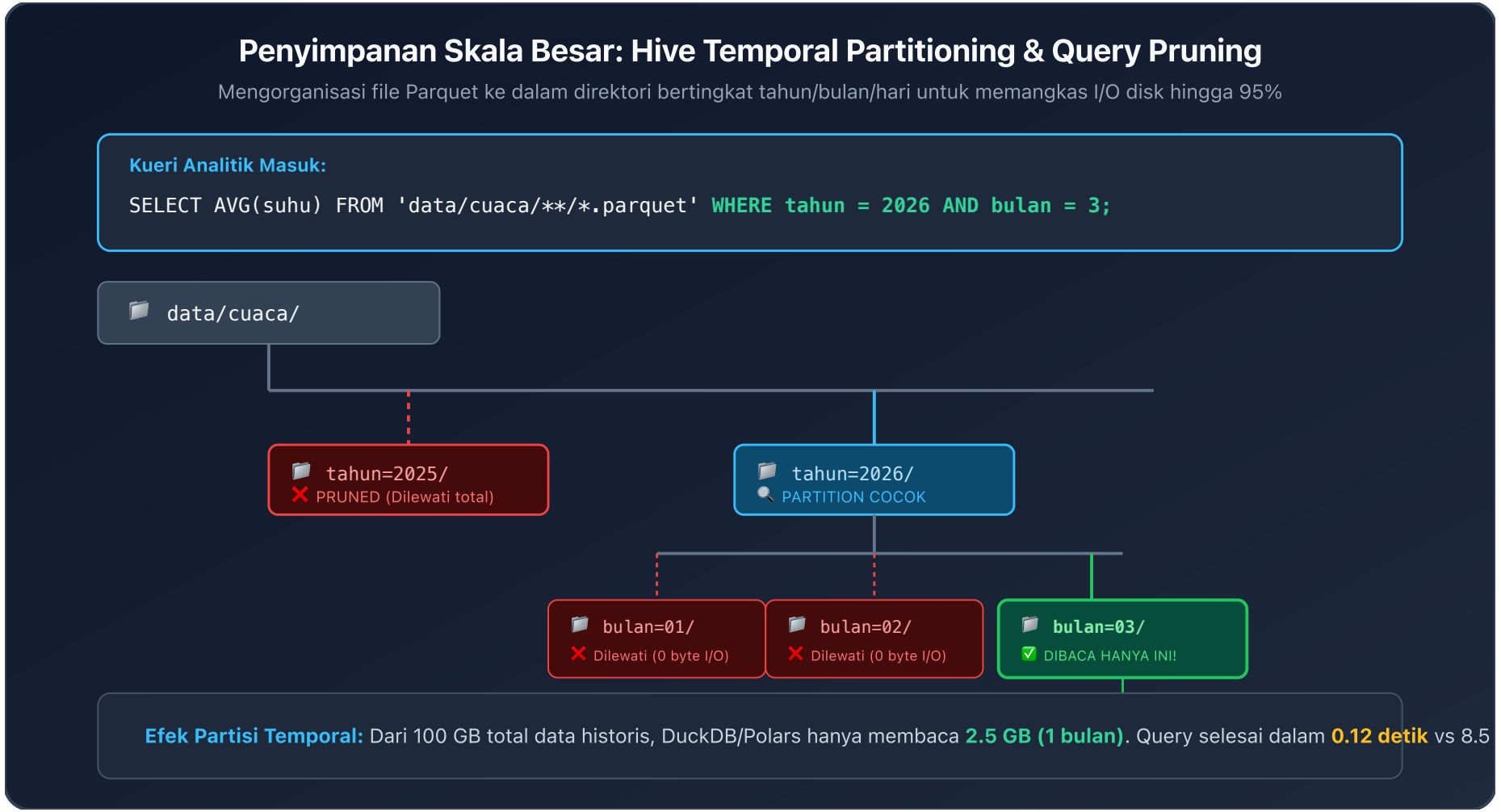
Fokus Evaluasi Checkpoint 2:

- **Soal 4:** Sintaks Polars untuk pengelompokan waktu berkala (`group_by_dynamic`).
- **Soal 5:** Perhitungan rata-rata bergerak jendela waktu (`rolling_mean`).
- **Soal 6:** Pembentukan fitur lag / selisih dengan nilai sebelumnya (`shift`).

> **Tips Analisis Kode:**
>
> - Perhatikan parameter `every` vs `period` pada agregasi berkala.
> - Pastikan kolom penanda waktu bertipe `pl.Datetime` dan terurut secara kronologis (`sort()`) sebelum dieksekusi.

Optimasi Big Data: Partisi Temporal Parquet

Menyimpan miliaran baris deret waktu dalam satu file raksasa adalah kesalahan fatal dalam arsitektur big data.



Menulis & Membaca Partisi Temporal di Polars & DuckDB

Struktur partisi direktori otomatis dikenali oleh mesin kueri modern (*Hive Partitioning*):

Menulis Partisi Hive via DuckDB:

```
# Ekspor dataframe ke partisi direktori
con.register("df_lengkap", df_lengkap)
con.execute("""
    COPY df_lengkap
    TO 'data/cuaca_partitioned'
    (FORMAT PARQUET,
     PARTITION_BY (tahun, bulan),
     OVERWRITE_OR_IGNORE 1);
""")
```

Struktur file di disk:

`data/cuaca_partitioned/tahun=2026/bulan=3/*.parquet`

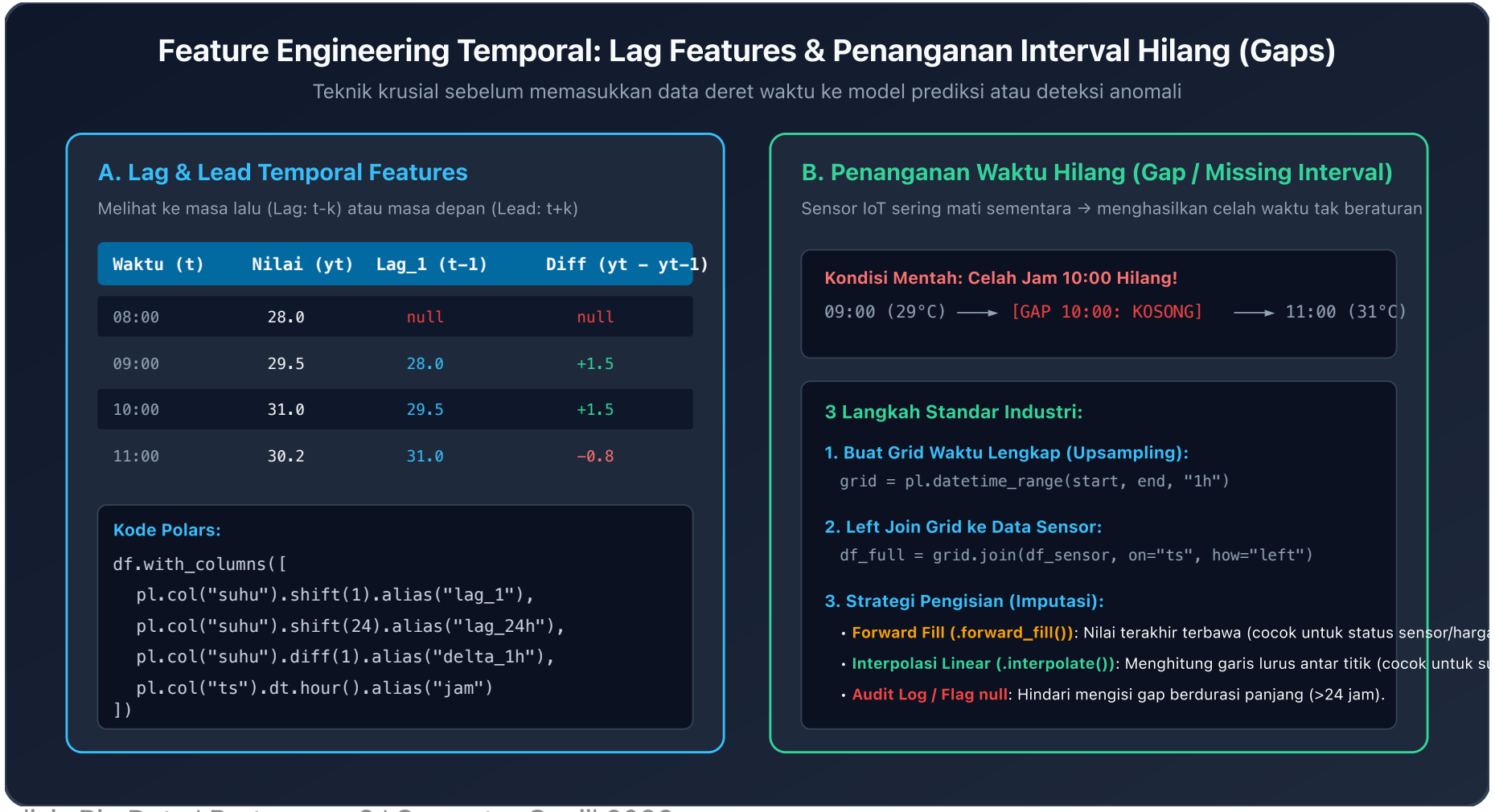
Kueri Efisien di DuckDB:

```
-- DuckDB hanya membaca folder bulan 3 tahun 2026
-- 95% file tahun lain langsung diabaikan (Pruning)
SELECT
    station_id,
    AVG(suhu) AS rata_suhu,
    MAX(kelembaban) AS max_lembab
FROM 'data/cuaca_partitioned/**/*.parquet'
WHERE tahun = 2026 AND bulan = 3
GROUP BY station_id;
```

> **Prinsip Kunci:** *Partition Pruning* memangkas biaya I/O disk dan mempercepat waktu eksekusi kueri dari hitungan menit menjadi milidetik.

Penanganan Interval Hilang (*Gaps*) pada Data Sensor

Sensor cuaca atau perangkat IoT di lapangan kerap mengalami kehilangan koneksi, menghasilkan jeda waktu kosong yang merusak perhitungan moving average atau model regresi.



Strategi Penanganan Gaps di Polars

```
# 1. Tentukan rentang waktu minimum dan maksimum sensor
t_start = df_sensor["timestamp"].min()
t_end = df_sensor["timestamp"].max()

# 2. Buat grid waktu lengkap tanpa ada jeda yang terlewat
grid_waktu = pl.DataFrame({
    "timestamp": pl.datetime_range(t_start, t_end, interval="1h", eager=True)
})

# 3. Gabungkan (Left Join) grid lengkap dengan data rekaman sensor riil
df_teratur = (
    grid_waktu
    .join(df_sensor, on="timestamp", how="left")
    .sort("timestamp")
    .with_columns([
        # Forward fill: nilai terakhir terbawa (cocok untuk status mesin)
        pl.col("status").forward_fill(),

        # Interpolasi linear: perkiraan garis lurus kontinu (cocok untuk suhu)
        pl.col("suhu").interpolate()
    ])
)
```

> **Aturan Integritas:** Jika jeda waktu sensor terlalu lama (misal: mati > 48 jam), jangan lakukan interpolasi! Berikan penanda bendera *anomali / sensor_rusak*.

Window Functions DuckDB untuk Time Series

DuckDB mendukung sintaks SQL analitik modern untuk operasi running total dan moving window secara native:

```
SELECT
    tanggal,
    kategori,
    penjualan_harian,
    -- 1. Running total akumulatif per kategori dari awal waktu
    SUM(penjualan_harian) OVER (
        PARTITION BY kategori
        ORDER BY tanggal
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS total_penjualan_kumulatif,

    -- 2. Rata-rata bergerak 3 hari (kemarin, hari ini, besok)
    AVG(penjualan_harian) OVER (
        PARTITION BY kategori
        ORDER BY tanggal
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) AS moving_avg_3_hari
FROM transaksi_harian;
```

> **Ingat:** `ROWS BETWEEN` menghitung jumlah baris fisik, sedangkan `RANGE BETWEEN` menghitung rentang nilai matematis waktu.

Checkpoint 3: Wayground Interaktif (Menit 85)

Checkpoint penutup untuk menguji kemampuan sintesis dan pemecahan kasus nyata:

Fokus Evaluasi Checkpoint 3:

- **Soal 7:** Keuntungan arsitektur *partitioning* berbasis tanggal pada Parquet.
- **Soal 8:** Langkah mitigasi data sensor cuaca yang memiliki jeda waktu kosong (*gap*).
- **Soal 9:** Kueri DuckDB *running total* menggunakan klausa `ROWS BETWEEN`.
- **Soal 10:** Rekayasa fitur kalender paling efektif dari kolom timestamp.

Refleksi Praktikal:

- Pilihan arsitektur penyimpanan (Partisi Parquet) berpengaruh langsung pada kecepatan kueri dan batas RAM.
- Fitur siklus (jam, hari, hari kerja) sering menjadi variabel terkuat dalam memprediksi fluktuasi beban sistem.

Praktik Hands-On Lab: Notebook Pertemuan 6

Buka JupyterLab dan jalankan notebook praktikal:

`notebooks/P06_timeseries/06_pemrosesan_timeseries.ipynb`

Studi Kasus Lab:

- **Dataset:** Simulasi 1 Juta Baris Data Sensor Cuaca Multi-Stasiun BMKG Indonesia (Suhu, Kelembaban, Curah Hujan, Kecepatan Angin).
- **Pipeline:**
  i. *Timestamp parsing & indexing*.
  ii. *Resampling* per jam & harian dengan `group_by_dynamic()`.
  iii. Pembuatan indikator *rolling statistics* (MA, Volatilitas).
  iv. Deteksi gap & interpolasi data sensor.
  v. Kueri running total dan ekspor partisi Parquet bertingkat.
  vi. Visualisasi tren & kalender interaktif dengan Plotly.

```
# Jalankan notebook di container
docker compose exec jupyter \
  python /home/jovyan/work/notebooks/\
P06_timeseries/06_pemrosesan_timeseries.py
```

Atau buka langsung file `.ipynb` di antarmuka web JupyterLab:

`http://localhost:8888/lab`

Pembaruan Tugas: Milestone 3 & Peluncuran Tugas 2

1. Tugas 1: Milestone 3 (Individual)

- **Fokus:** Penerapan analisis deret waktu atau spasial pada dataset pilihan mahasiswa (>1M baris / >500MB).
- **Deliverables:**
  - Analisis pola temporal (Polars dynamic/rolling window atau DuckDB window function).
  - Minimal 6 visualisasi interaktif Plotly/Altair baru.
- **Target Check-in:** Pertemuan 8 di GitHub Classroom.

2. Tugas 2: Phase 1 Pitch & Arsitektur (Tim)

- **Tipe:** Proyek Kelompok (2–3 Mahasiswa).
- **Fokus:** Membangun *Data Product* berbasis dashboard interaktif Streamlit + DuckDB/Polars yang di-hosting di HuggingFace Spaces.
- **Phase 1 Deliverables:**
  - Pembentukan tim di GitHub Classroom.
  - Penulisan proposal topik dan skema arsitektur data di `docs/architecture.md`.
- **Target Check-in:** Pertemuan 7.

Rangkuman Materi Pertemuan 6

1. **Struktur Deret Waktu:** Terdiri atas *Trend* (arah jangka panjang), *Seasonality* (variasi berulang periodik), dan *Noise* (fluktuasi residu acak).
2. **Resampling Efisien:** `group_by_dynamic()` di Polars membagi data ke dalam *bucket* interval diskrit dengan kecepatan eksekusi tinggi untuk *downsampling* data masif.
3. **Windowing Halus:** `rolling_mean()` dan `rolling_std()` menghitung statistik lokal pada jendela bergerak tanpa mengurangi jumlah baris data.
4. **Feature Engineering:** Fitur *Lag* (`shift(1)`) dan variabel kalender (jam, hari, hari kerja) membentuk fondasi utama analisis temporal.
5. **Skala Besar:** *Temporal Partitioning* (`year=YYYY/month=MM`) memangkas I/O kueri secara radikal melalui *partition pruning*.
6. **Integritas Data:** Celah waktu (*gap*) wajib diselaraskan menggunakan grid waktu lengkap sebelum dilakukan interpolasi atau imputasi.

Terima Kasih & Selamat Berpraktik!

Silakan lanjutkan ke notebook praktikum:

`notebooks/P06_timeseries/06_pemrosesan_timeseries.ipynb`

Ada pertanyaan seputar Milestone 3 atau Tugas 2? Diskusikan sekarang di kelas!